Operating Systems: SEng 3207

Chapter Three
Memory Management

Lecture 7- Main Memory

ASTU
Department of CSE

OUTLINE

- ✓ **Background**
- ✓ **Swapping**
- ✓ **Contiguous Allocation**
- ✓ **Paging**
- ✓ **Segmentation**

Background

- Program must be brought into memory and placed within a process for it to be executed.
- **Input Queue** - collection of processes on the disk that are waiting to be brought into memory for execution.
- Main memory and registers are only storage CPU can access directly.
- Register access in one CPU clock (or less)
- Main memory can take many cycles
- **Cache** sits between main memory and CPU registers
- Memory needs to be allocated efficiently to pack as many processes into memory as possible.
- **Problem**
  - How to manage relative speed of accessing physical memory
  - How to Ensure correct operation to protect the operating system from being accessed by user process and user processes from one another.

One possible solution for the above problem

- Make sure that each process has **a separate address space.**
  - ✓ Determine the legal addresses that the process can access legally
  - ✓ Ensure that a process can access only these legal addresses.
- This protection can be done using **two registers**
  - **Base registers**: - holds the smallest legally accepted physical address.
  - **Limit registers**:-specifies the range.

Defines legal addresses

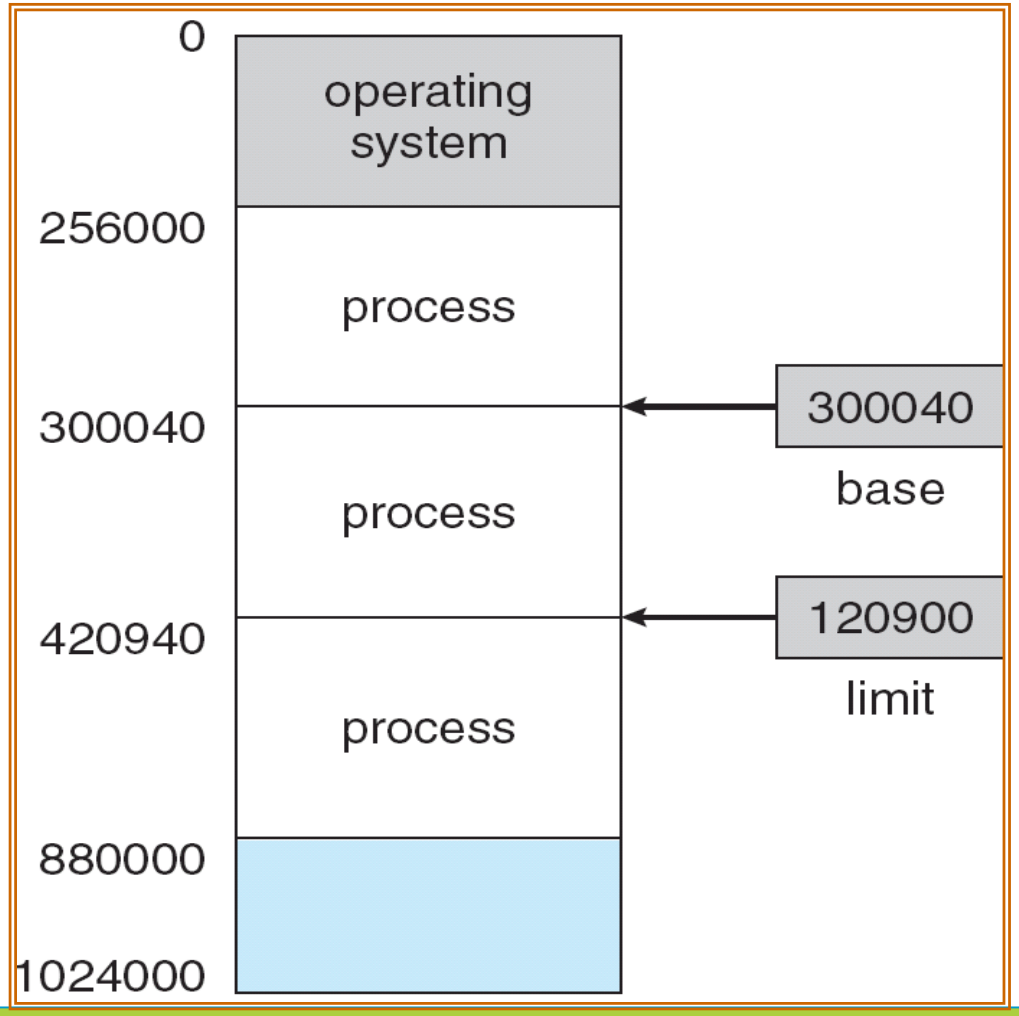
- Use a HW to compare every address generated in user space with registers value

Ensures memory protection

- A trap is generated for any attempt by a user process to access beyond the limit.
- Base and limit registers are loaded only by the OS, using special privileged instructions(**what's its advantage?**)

Base and Limit Registers

- A pair of **base** and **limit** registers define the logical address space



300040+120900=

Valid address=[300040,420940]

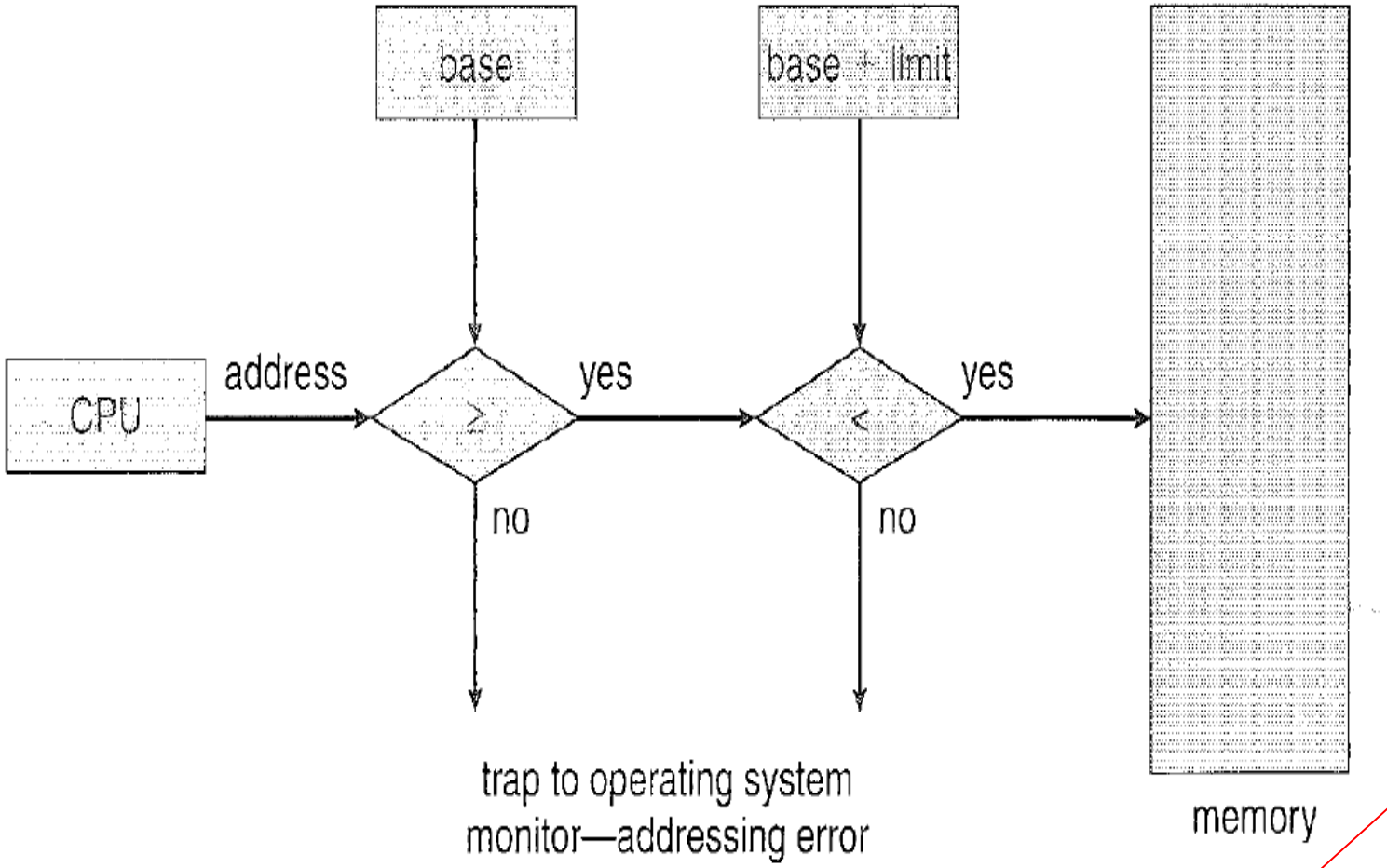


Figure | Hardware address protection with base and limit registers.

Can you explain what this figure depicts?

Address Binding

- Usually a program resides in a disk in the form of executable binary file.
- It is brought to memory for execution (it may be moved between disk and memory in the meantime).
- When a process is executed it accesses instructions and data from memory. When execution is completed the memory space will be freely available.
- A user program may be placed at any part of the memory.
- A user program passes through a number of steps before being executed.
- Addresses may be represented in different ways during these steps.
  - ✓ **Symbolic addresses**:- addresses in source program(eg count)
  - ✓ ***Re-locatable*** addresses:(eg. 14 bytes from the beginning of this module)
  - ✓ ***Absolute addresses***:-(eg. 74014)

What's the purpose of binding?

Binding of instructions and data to memory

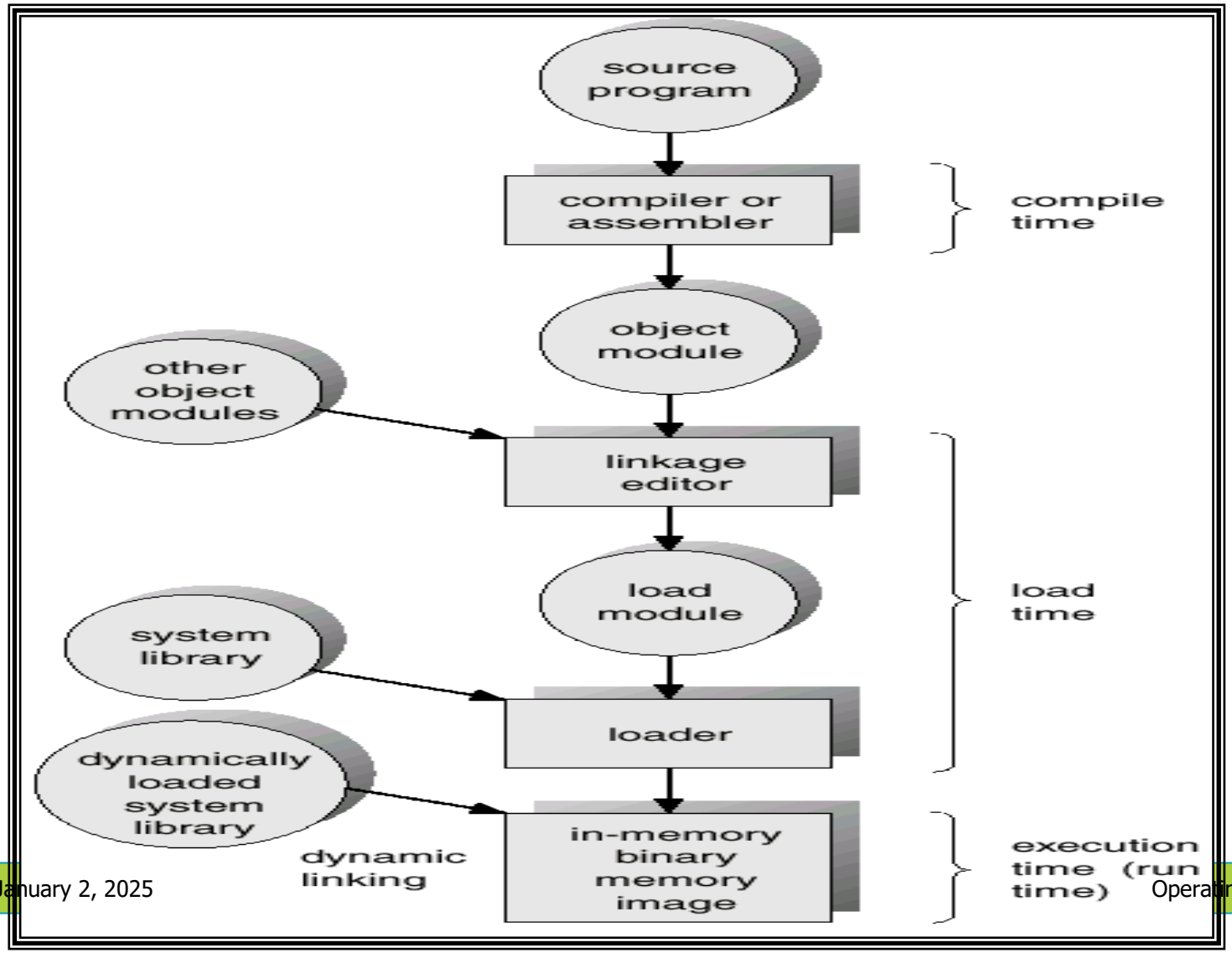
– **Address binding of instructions and data to memory addresses can happen at three different stages.**

- **Compile time:**
  - If memory location is known a prior, absolute code can be generated; must recompile code if starting location changes.
- **Load time:**
  - Must generate re-locatable code if memory location is not known at compile time.
- **Execution time:**
  - Binding delayed until runtime if the process can be moved during its execution from one memory segment to another.
  - Need hardware support for

address maps (e.g. base and limit registers).

What are the tradeoffs in these stages?

Multi-step Processing of a User Program



Logical vs. Physical Address Space

- The concept of **a logical address space** that is bound to **a separate physical** address space is central to proper **memory management**.
  - **Logical Address:** or virtual address - generated by CPU
    - Set of logical addresses generated by a program is **called logical address space.**
  - **Physical Address:** address seen by memory unit.
    - Set of physical addresses corresponds to logical space is called **physical address space**

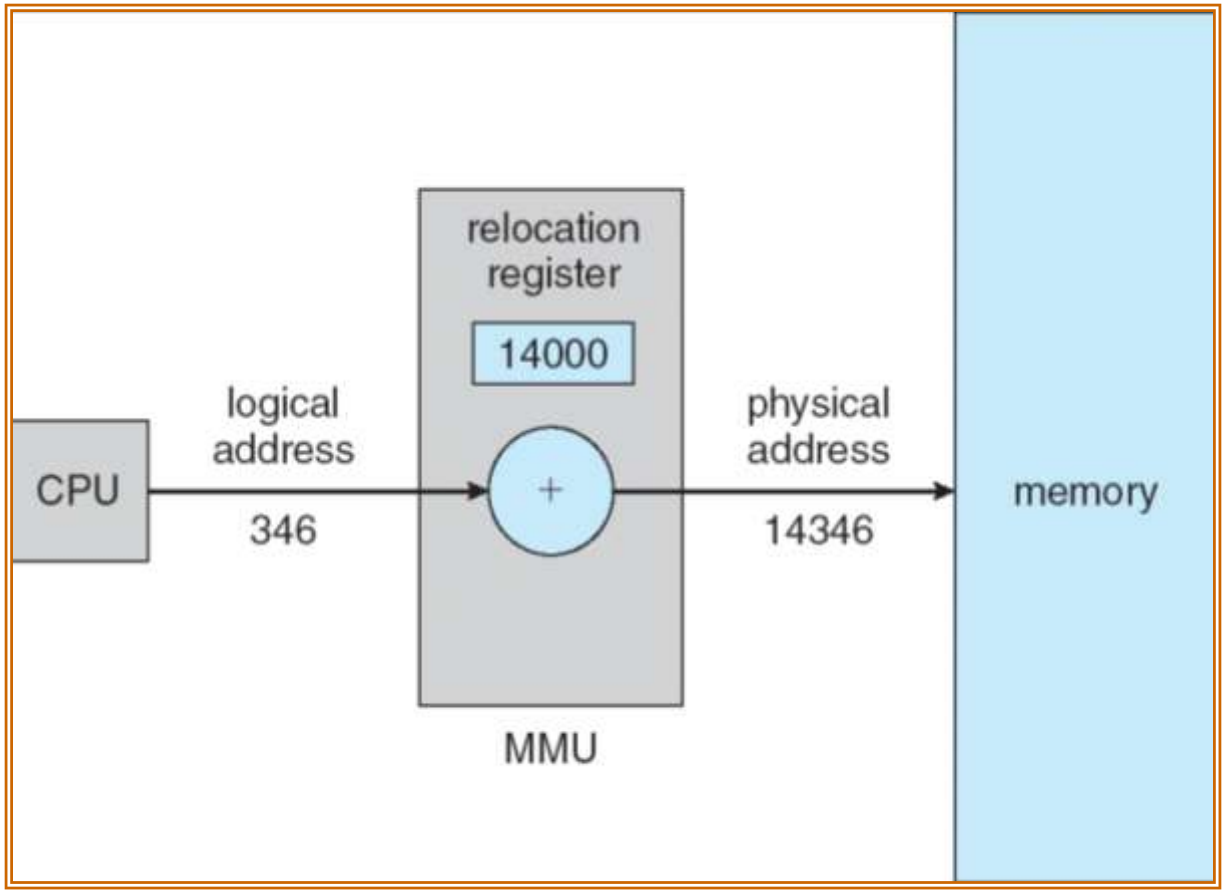
What's the difference between logical and physical addresses?

- **Logical and physical addresses** are the same in compile-time and load-time address-binding schemes; **logical (virtual) and physical addresses** differ in execution-time address-binding scheme.

Memory Management Unit (MMU)

- Hardware device that **maps virtual to physical address**.
- In MMU scheme, the value in the relocation register is added to every address generated by a user process at the time it is sent to memory.
- The user program deals with logical addresses; it never sees the real physical address.

Dynamic relocation using a relocation register



Can you explain what this figure depicts?

Dynamic Loading

- In the discussion made so far, we assume that a program and its associated resources must be loaded to memory before execution(not efficient in resource utilization).
- **But In Dynamic loading**
  - Routine is not loaded until it is called.
  - Better memory-space utilization; unused routine is never loaded.
  - Useful when large amounts of code are needed to handle infrequently occurring cases.
  - No special support from the operating system is required; implemented through program design.

Dynamic Linking

- Linking postponed until execution time.
- Small piece of code, ***stub***, used to locate the appropriate memory-resident library routine.
- Stub replaces itself with the address of the routine, and executes the routine.
- Operating system needed to check if routine is in processes' memory address.
- Dynamic linking is particularly useful for libraries
- System also known as **shared libraries**

What's an overlay?

Overlays

- Keep in memory only those instructions and data that are needed at any given time.
  - Needed when process is larger than amount of memory allocated to it.
  - Implemented by user, no special support from OS; programming design of overlay structure is complex.

Swapping

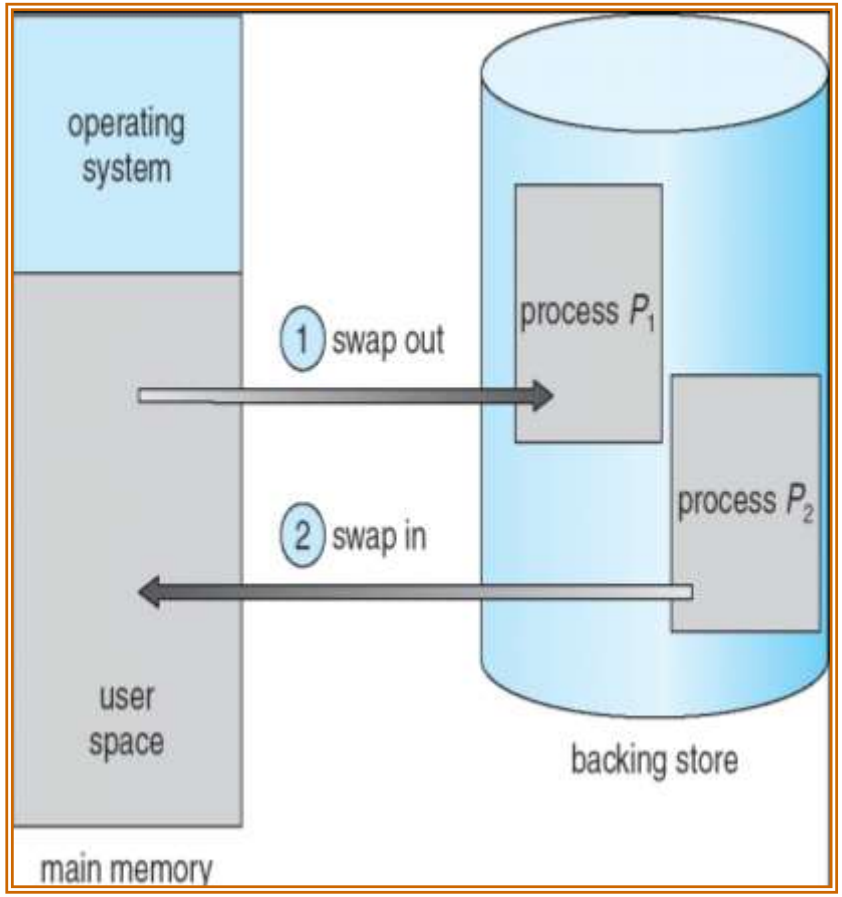
- **A process can be swapped temporarily out of memory to a backing store and then brought back into memory for continued execution.**
  - **Backing Store** - fast disk large enough to accommodate copies of all memory images for all users; must provide direct access to these memory images.
  - **Roll out, roll in** - swapping variant used for priority based scheduling algorithms; lower priority process is swapped out, so higher priority process can be loaded and executed.
  - Major part of swap time is transfer time; total transfer time is directly proportional to the amount of memory swapped(**might be?).**
    - In swapping
      - context time must be taken into consideration.
      - Swapping a pending process for an IO needs care.
  - Modified versions of swapping are found on many systems, i.e. UNIX and Microsoft Windows.
    - System maintains a **ready queue** of ready-to-run processes which have memory images on disk.
    - Give possible solution

- Why is swapping needed?
- What issues need to be considered in swapping?

Schematic View of Swapping

Swapping is done when

- Quantum expires
- Priority issues arises

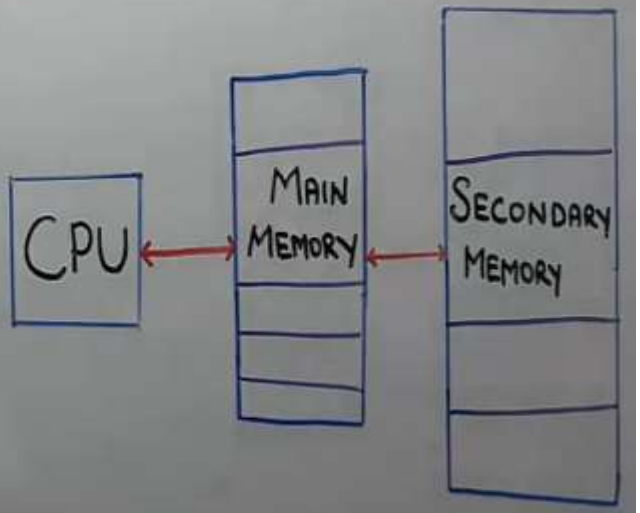


Conditions for swapping

- Process to be executed must be not in memory
- No sufficient free memory location

B.TECH/B.E-BCA-MCA-CSE-IT
GATE-UGC-NET-PSU UNIV.E

DEF:- MEMORY MANAGEMENT IS A FUNCTIONALITY OF OPERATING SYSTEM WHICH MANAGES THE 'MAIN MEMORY' AND KEEP TRACK OF PROCESSES MOVING FROM 'SECONDARY MEMORY TO MAIN MEMORY TO CPU' AND VISE VERSA.



FUNCTIONALITY:-

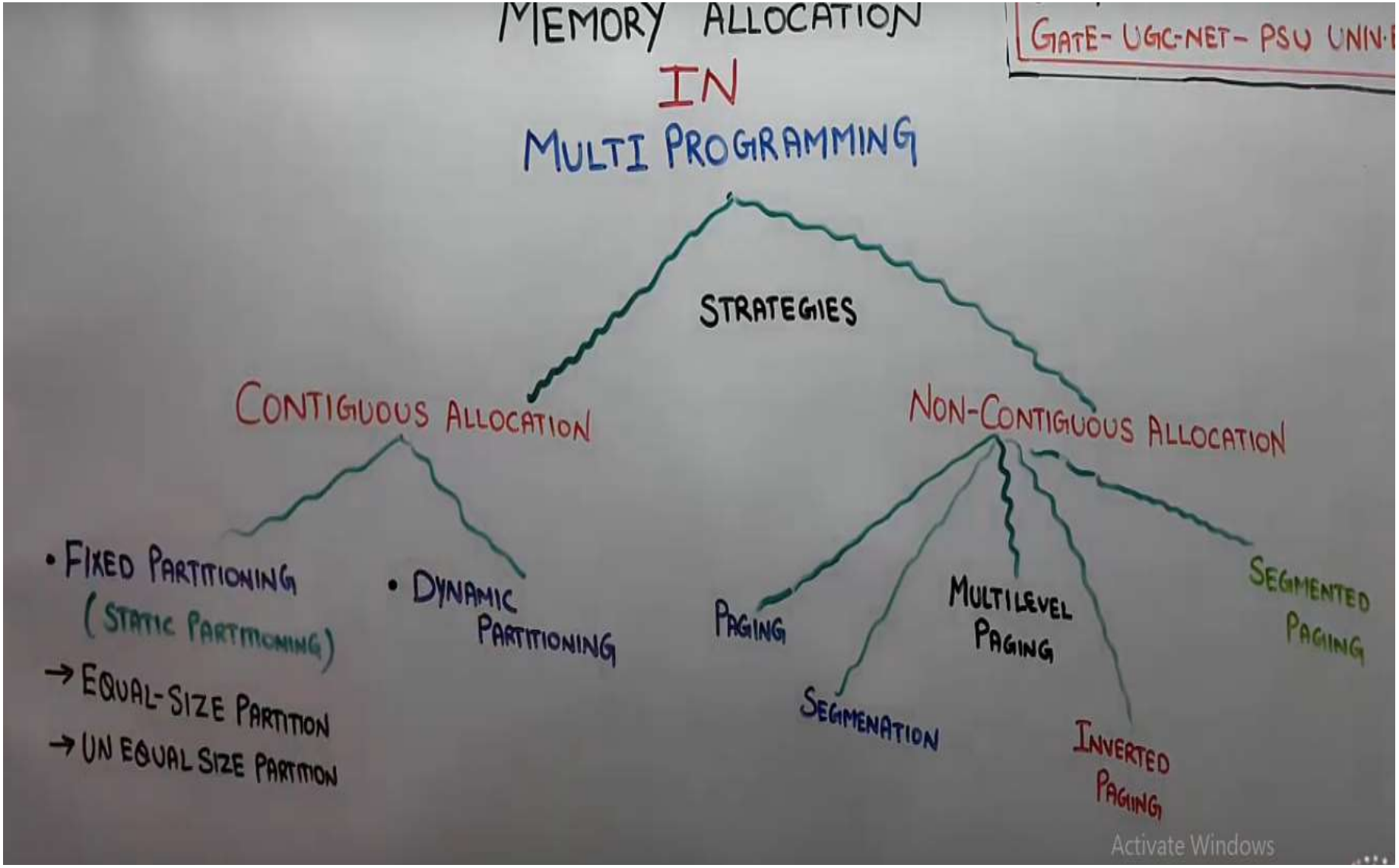
- KEEP TRACK OF WHAT 'MEMORY' IS IN 'USE' AND WHAT MEMORY IS 'FREE'
- ALLOCATE FREE MEMORY TO PROCESSES WHEN 'NEEDED' AND
  - → 'DEALLOCATES' IT WHEN THEY DONOT REQUIRED
- MANAGING THE TRANSFER OF 'MEMORY' 'RAM & DISK'
- PROHIBITING USER PROGRAM TO ENTER IN OS AREA & OTHERS USER AREA
- KEEP UPDATING STATUS OF PROCESSES

GOALS:-

- MAXIMISE CPU UTILISATION
- MINIMISE RESPONSE TIME

→ MAXIMISE MEMORY UTILISATION
→ PRIORITISE IMPORTANT PROCESSES
→ SECURITY TO USER PROGRAMS AND OPERATING SYSTEM

Activate Windows
Go to Settings to activate Windows.


MEMORY ALLOCATION
IN
MULTI PROGRAMMING
GATE- UGC-NET- PSU UNIV
STRATEGIES
CONTIGUOUS ALLOCATION
NON-CONTIGUOUS ALLOCATION
• FIXED PARTITIONING
(STATIC PARTITIONING)
→ EQUAL-SIZE PARTITION
→ UN EQUAL SIZE PARTITION
• DYNAMIC PARTITIONING
PAGING
SEGMENATION
MULTILEVEL PAGING
INVERTED PAGING
SEGMENTED PAGING
Activate Windows

CONTIGUOUS MEMORY ALLOCATION

GATE- UGC-NET- PSU UNIV-EX

FIXED PARTITIONING

CONTIGUOUS MEMORY ALLOCATION:- SETTING UP THE PARTITION BOUNDARIES OF & ALLOCATE ONE SINGLE 'CONTIGUOUS' BLOCK OF MEMORY TO A SINGLE PROCESS.

FIXED PARTITIONING: (STATIC PARTITIONING)
- → FIXED NO. OF PARTITION (AT CONFIGURATION TIME)
- → BOUNDARIES OF PARTITIONS ARE FIXED (CAN'T MOVE)

ALSO KNOWN AS 'MFT - MULTIPROGRAMMING WITH FIXED NO. OF TASKS)

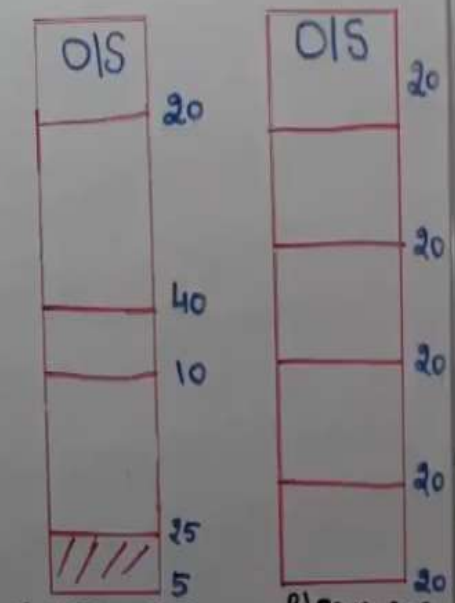
→ SPANNING (CANNOT BREAK THE PROCESS) IS NOT ALLOWEDED

FRAGMENTATION: WASTAGE OF MEMORY

- FRAGMENTATION IS OF TWO TYPES:-

1) **INTERNAL FRAGMENTATION:** THAT FRAGMENTATION WHICH IS INTERNAL PART OF ALLOCATED PARTITION.

2) **EXTERNAL FRAGMENTATION:** FRAGMENTATION DUE TO UNALLOCATED PARTITION.



A) UNEQUAL SIZE PARTITION

B) EQUAL SIZE PARTITION

Activate Windows
Go to Settings to activate Windows

Contiguous Allocation

- Main memory usually divided into two partitions
  - Resident Operating System, usually held in low memory with interrupt vector.
  - User processes then held in high memory.
- **Single partition allocation**
  - **Relocation register scheme used to protect user processes from each other, and from changing OS code and data.**
  - **Relocation register** contains value of smallest physical address; limit register contains range of logical addresses - each logical address must be less than the limit register.
  - When CPU scheduler selects a process for execution, the dispatcher loads the relocation and limit registers with the correct values. Every address generated by CPU is compared against these values.

  ➔ensures memory protection of OS and other processes from being modified by the running process.

Contiguous Allocation (cont.)

- **Multiple partition Allocation**
  - Hole - block of available memory; holes of various sizes are scattered throughout memory.
  - When a process arrives, it is allocated memory from a hole large enough to accommodate it.
  - Operating system maintains information about which partitions are
    - allocated partitions
    - free partitions (hole)

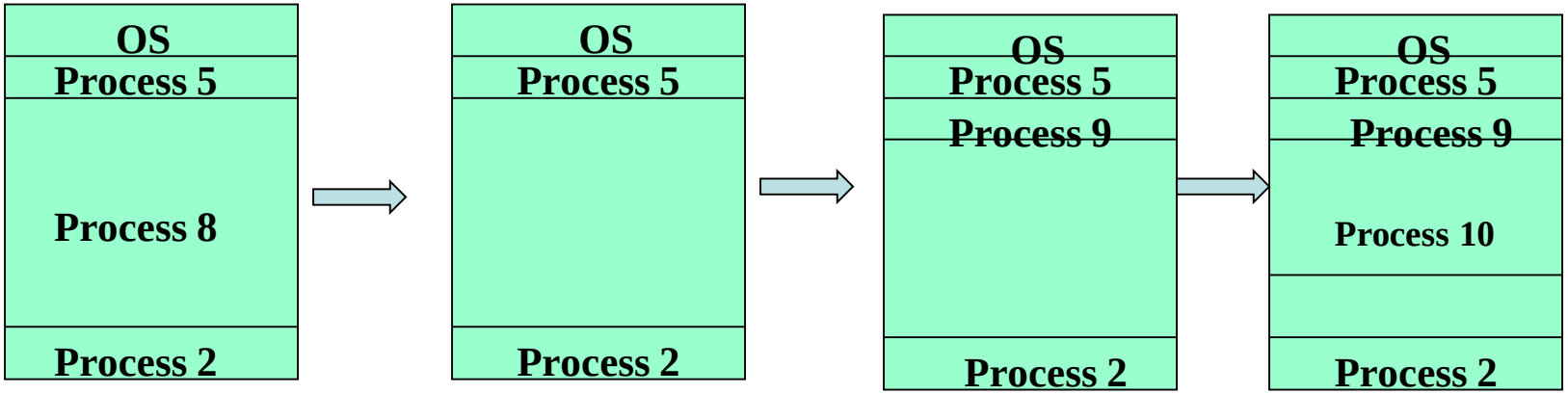


Fig :Contiguous Allocation example

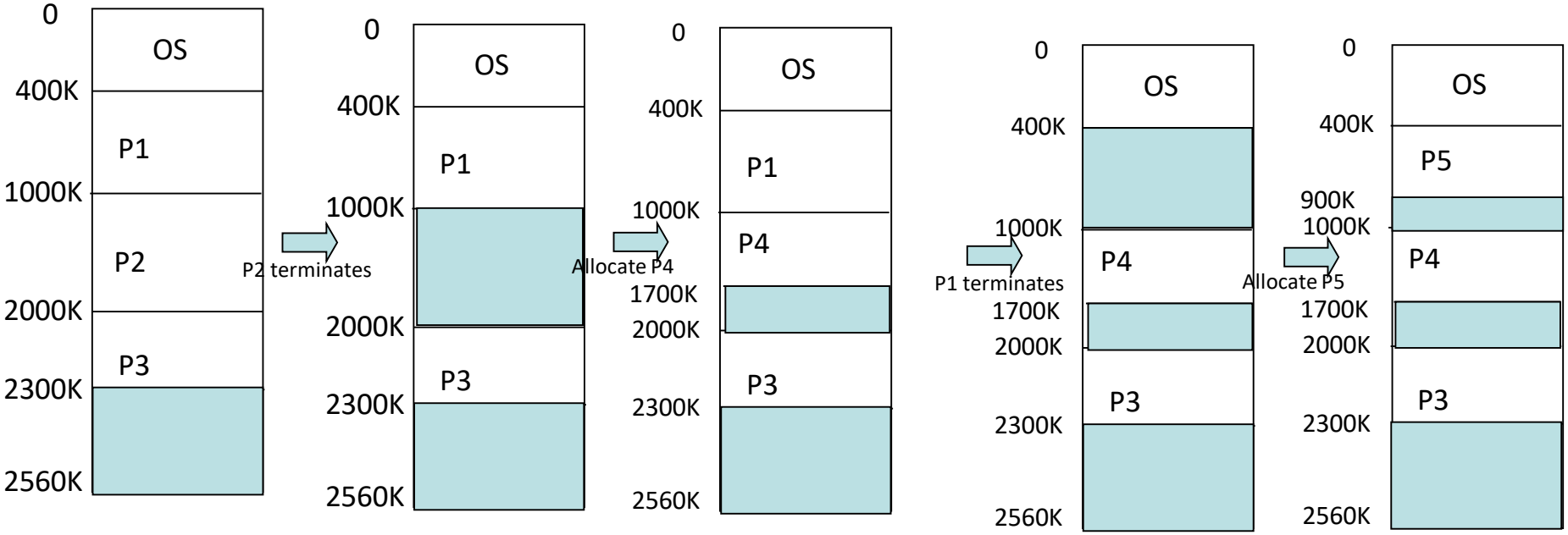
•Dynamic Partitioning

- Partitions are of variable length and number
- Process is allocated exactly as much memory as required
- Eventually get holes in the memory. **This is called external fragmentation**
- Must use compaction to shift processes so they are contiguous and all free memory is in one block

Contiguous Allocation (Cont.)

- Example: 2560K of memory available and a resident OS of 400K. Allocate memory to processes P1…P4 following FCFS.
- **Shaded regions are holes**
- Initially P1, P2, P3 create first memory map.

Process	Memory	time
P1	600K	10
P2	1000K	5
P3	300K	20
P4	700K	8
P5	500K	15



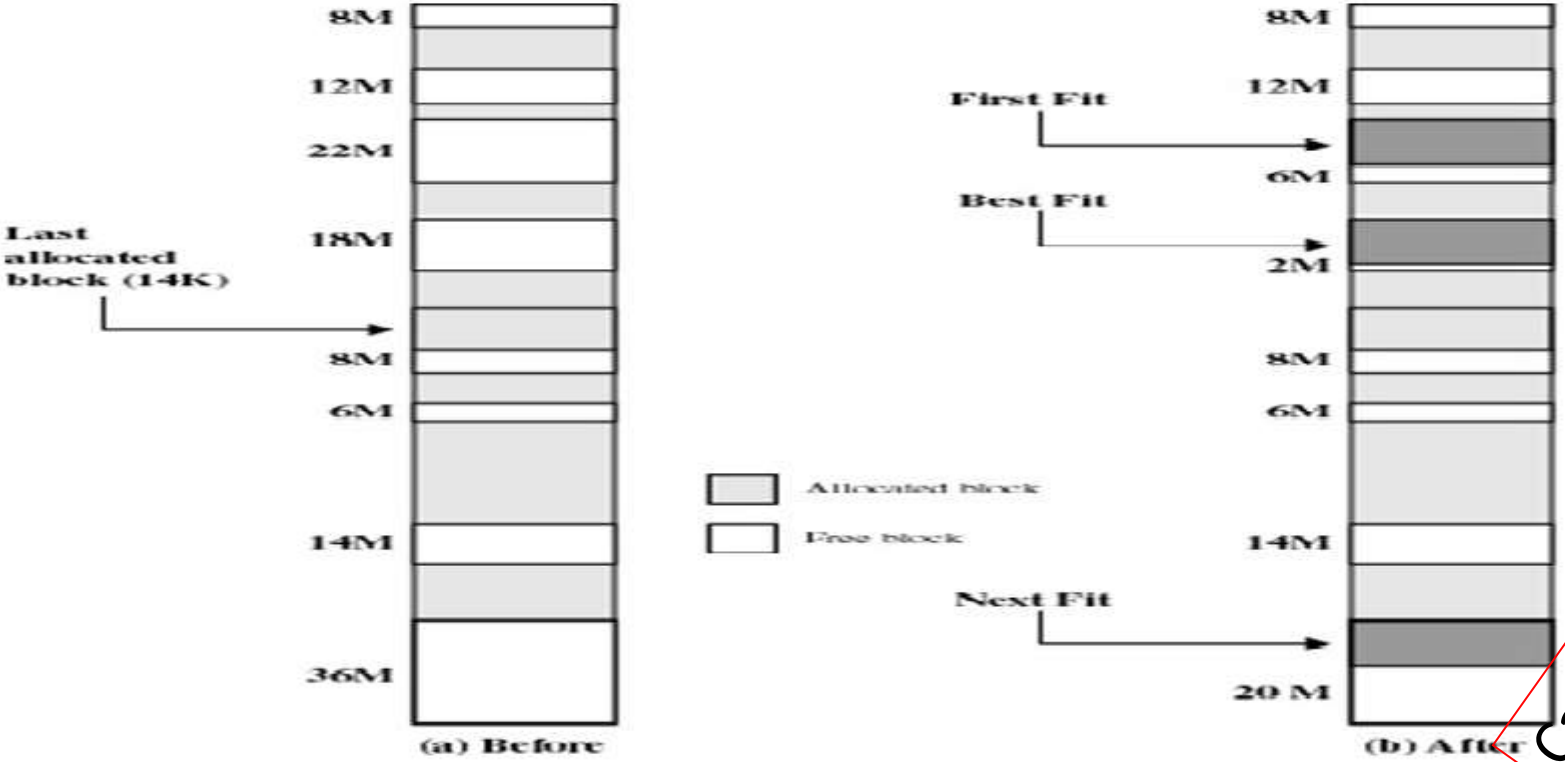
Dynamic Storage Allocation Problem

- Operating system must decide which free block to allocate to a process.
- How to satisfy a request of size n from a list of free holes-use algorithms

- **First-fit**
  - allocate the first hole that is big enough
  - Fastest
- **Best-fit**
  - Allocate the smallest hole that is big enough; must search entire list, unless ordered by size. Produces the smallest leftover hole.
  - the smallest amount of fragmentation is left
  - Worst performer overall
- **Worst-fit**
  - Allocate the largest hole; must also search entire list.
  - Produces the largest leftover hole.
  - **First-fit and best-fit are better than worst-fit in terms of speed and storage utilization.**

Which algorithm has the least scanning overhead? Why?

Dynamic Storage Allocation Problem



Can you describe this figure?

Figure | Example Memory Configuration Before and After Allocation of 16 Mbyte Block

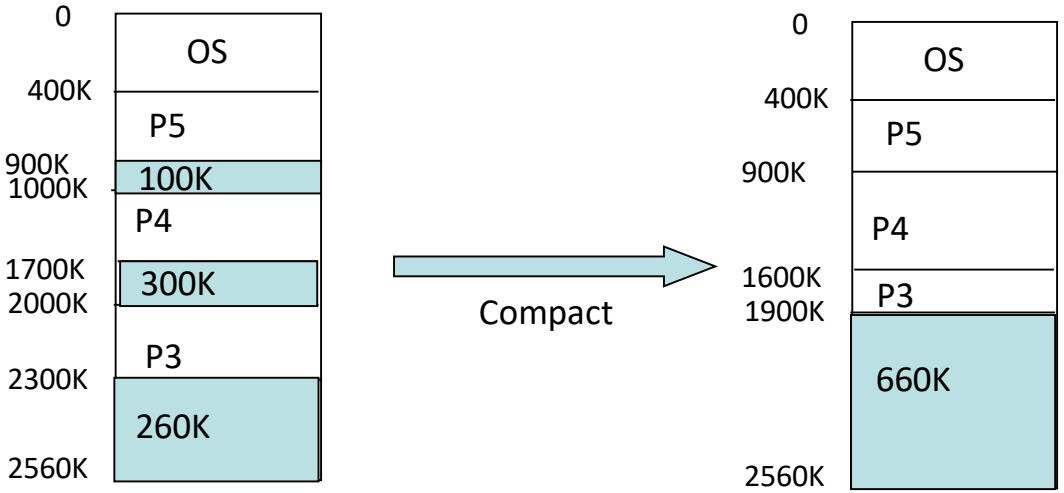
Fragmentation

- **External Fragmentation** – total memory space exists to satisfy a request, but it is not contiguous.
  - 50 % rule: Given N allocated blocks 0.5 blocks will be lost due to fragmentation.
- **Internal Fragmentation** – allocated memory may be slightly larger than requested memory; this size difference is memory internal to a partition, but not being used.
  - Consider the hole of 18,464 bytes and process requires 18462 bytes.
  - If we allocate exactly the required block, **we are left with a hole of 2 bytes.**
  - The overhead to keep track of this free partition will be substantially larger than the hole itself.
  - **Solution**: allocate very small free partition as a part of the larger request.

Solution to fragmentation

1. **Compaction**
2. **Paging**
3. **Segmentation**

1. **Compaction:** Reduce external fragmentation by compaction
   - Shuffle memory contents to place all free memory together in one large block.
   - Compaction is possible *only* if relocation is dynamic, and is done at execution time.
   - Compaction depends on cost.



2. Paging

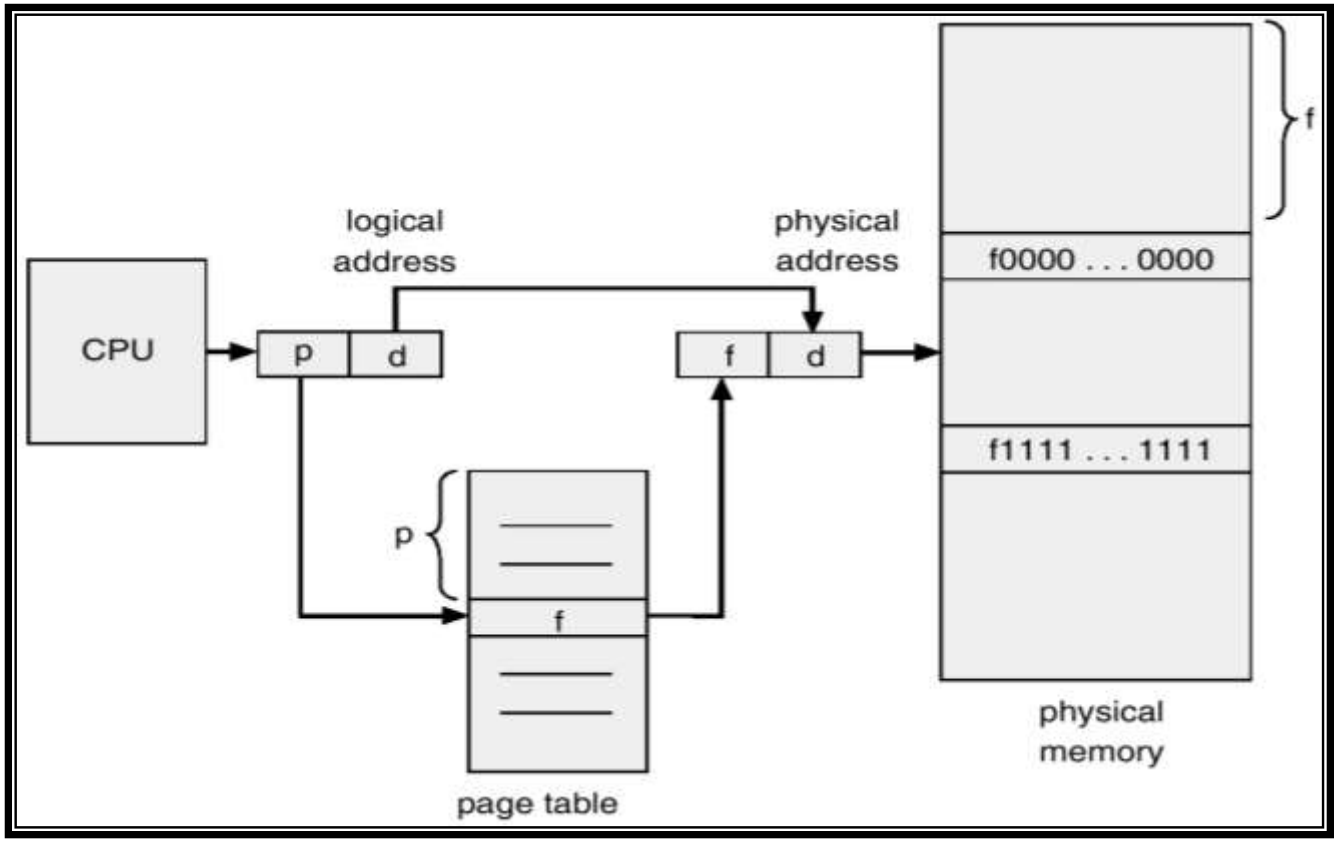
- **Basic Idea:** logical address space of a process can be made noncontiguous; process is allocated physical memory wherever it is available.
- Divide physical memory into fixed-sized blocks called **frames** (size is power of 2, between 512 bytes and 8,192 bytes)
- Divide logical memory into blocks of same size called **pages**
- Keep track of all free frames
- To run a program of size ***n*** pages, need to find *n* free frames and load program
- Set up a page table to translate logical to physical addresses
    - ***Note:: Internal Fragmentation possible!!***
- Paging is a form of dynamic relocation(**Explain why?**)

Address Translation Scheme

- **Address generated by CPU is divided into:**
  - ***Page number (p)*** – used as an index into a *page table* which contains base address of each page in physical memory.
  - ***Page offset (d)*** – combined with base address to define the physical memory address that is sent to the memory unit.
- Page number is an index to the page table.
- The page table contains base address of each page in physical memory.
- The base address is combined with the page offset to define the physical address that is sent to the memory unit.
- The size of a page is typically a power of 2.
  - 512 –8192 bytes per page.
- The size of logical address space is 2^m and page size is 2^n address units.
- Higher m-n bits designate the page number
- n lower order bits indicate the page offset.

Page #	p	d	Page offset
	m-n	n	

Address Translation Architecture



Explain what is depicted in the figure?

Paging Example(cont….)

Assume:-

- page size=4 bytes
- physical memory = 32 bytes (8 pages).

How a logical memory address can be mapped into physical memory?

✓ Logical address 0(containing '**a**')

i. is on page 0.

ii. Is at offset 0.

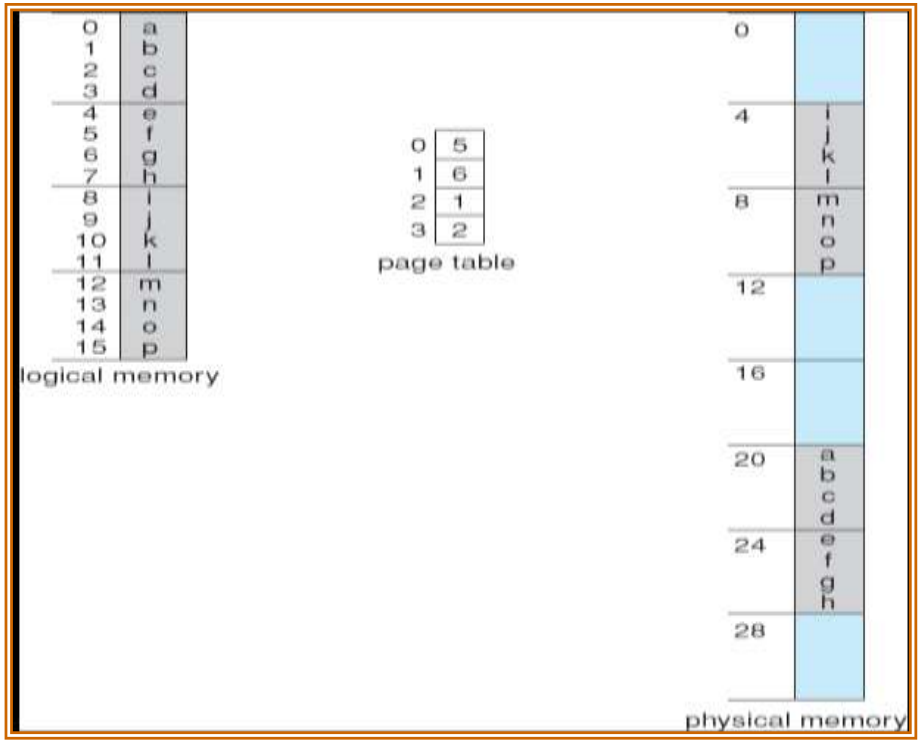
✓ Indexing into the page table, you can that page 0 is in frame 5.

➔ logical address 0 is mapped to ***physical 20***, i.e.

✓ 20=[(5x4)+0]

Similarly,

a. Logical address 13(page 3,offset 2) mapped ➔physical address 9=((2x4)+1).



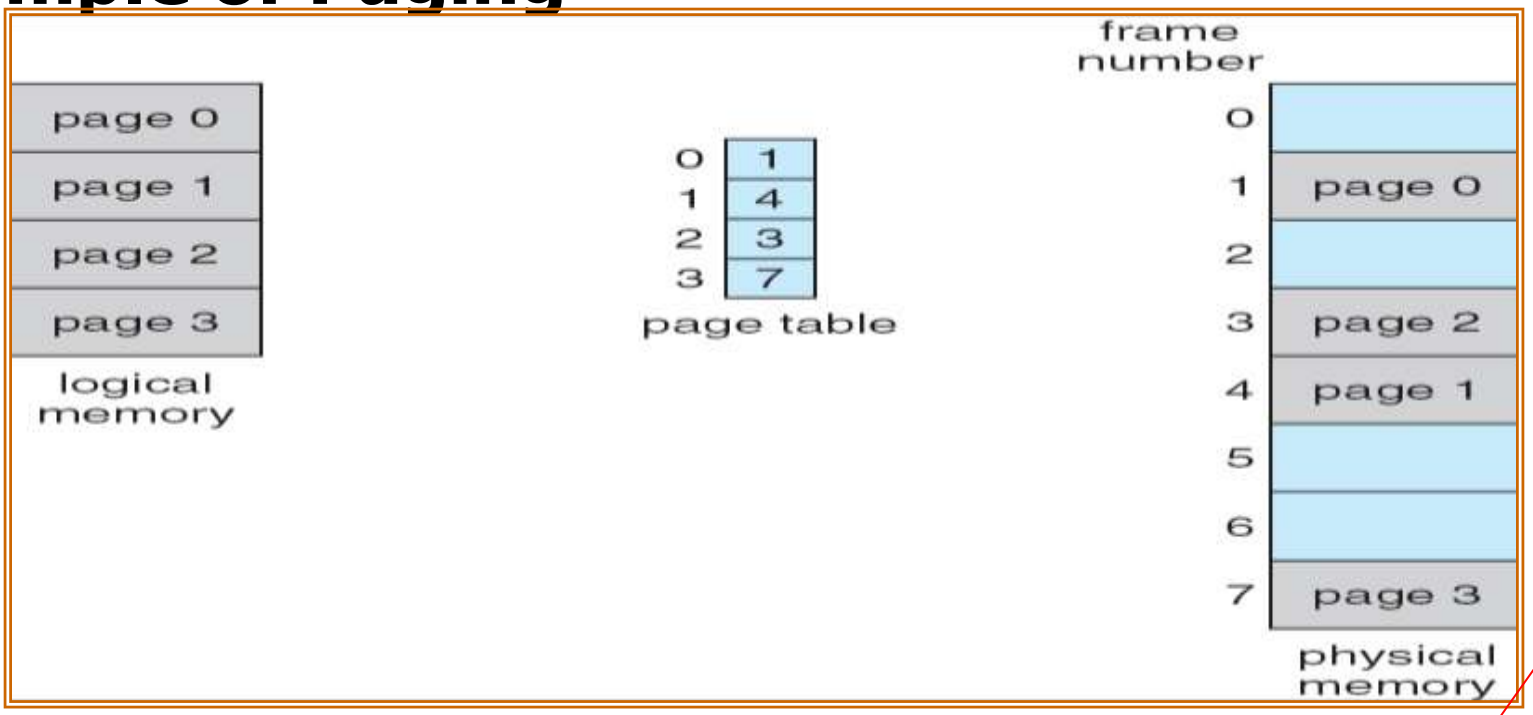
✓ Logical address 0 maps 5*4+0=20

✓ Logical address 3 maps to= 5*4+3=23

✓ Logical address 4 maps to =6*4+0=24

✓ Logical address 13 maps to= 2*4+1=9.

Example of Paging



- **Page size= 4 bytes; Physical memory=32 bytes (8 pages)**
- Logical address 0 maps 1*4+0=4
- Logical address 3 maps to= 1*4+3=7
- Logical address 4 maps to =4*4+0=16
- Logical address 13 maps to= 7*4+1=29.

Can you describe this figure?

More on Paging(contd.)

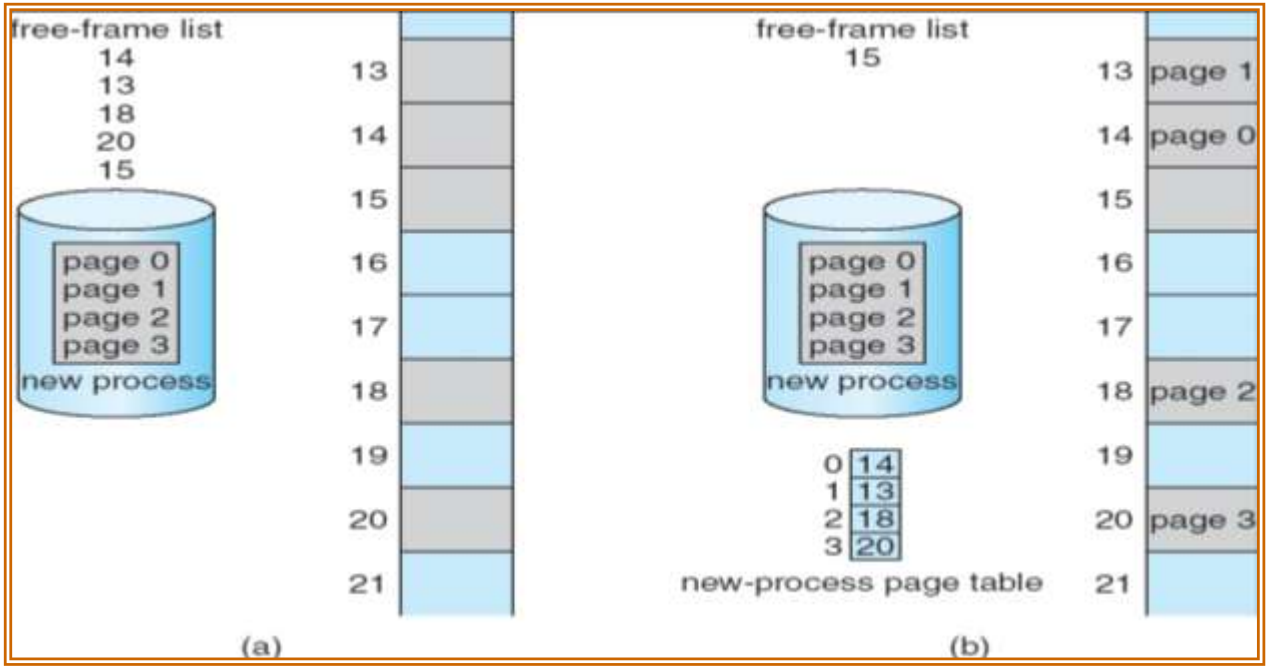
- In paging scheme, Pages are allocated as units.
- In this scheme there is no external fragmentation.
- But internal fragmentation is inevitable.
- **Example:-**
  - assume page size=2048 bytes.
  - a process of 61442 bytes needs 30 pages plus 2 bytes. Since units are managed in terms of pages **31** pages are allocated.
  - Internal fragmentation=2046 bytes!!!!.
- In the worst case a process needs n pages plus 1 byte. So it will be allocated n+1 pages.

➔Fragmentation =(page size-1 byte) ~ entire page.

In paging scheme, is there any chance for user process to enter an address of the OS or an other user process ? Why?

Free Frames

- When a process arrives the size in pages is examined
- Each page of process needs one frame.
- If n frames are available these are allocated, and page table is updated with frame number.



Before allocation

After allocation

Can you describe this figure?

Implementation of Page Table

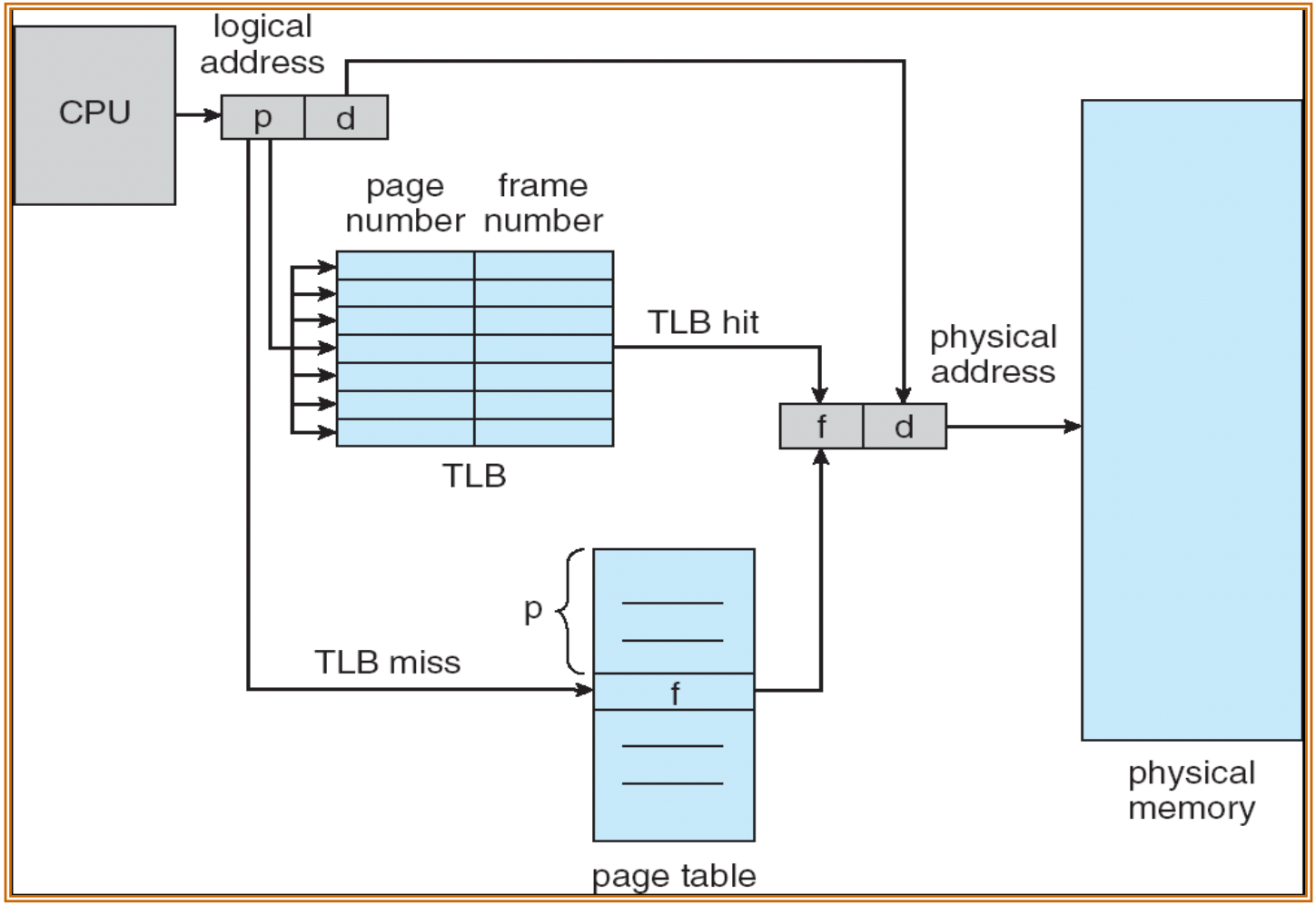
- Two options: Page table can be kept in registers or main memory
- Page table is kept in main memory due to bigger size.
  - Ex: address space = 2^{32} to 2^{64}
  - Page size= 2^{12}
  - Page table= 2^{32} / 2^{12}=2^{20}
  - If each entry consists of 4 bytes, the page table size = 4MB.
- ***Page-table base register (*PTBR) points to the page table.**
- ***Page-table length register* (PTLR) indicates size of the page table.**
- **PTBR, and PTLR are maintained in the registers.**
- Context switch means changing the contents of PTBR and PRLR.
- In this scheme every data/instruction access requires two memory accesses. One for the page table and one for the data/instruction.
  - Memory access is slowed by a factor of 2.
- The two memory access problem can be solved by the use of a special fast-lookup hardware cache called *associative memory* or *translation look-aside buffers (TLBs)*

Translation look-aside buffer (TLB)

- **TLB is an associative memory – parallel search**
- A set of associative registers is built of especially high-speed memory.
- Each register consists **of two parts: key and value**
- When associative registers are presented with an item, it is compared with all keys simultaneously.
- If corresponding field is found, corresponding field is output
- Associative registers contain only few of page table entries.
  - When a logical address is generated it is presented to a set of associative registers.
  - If yes, the page is returned.
  - Otherwise memory reference to page table mode. Then that page # is added to associative registers.
- Address translation (A´, A´´)
  - If A´ is in associative register, get frame # out.
  - Otherwise get frame # from page table in memory
- It may take 10 % longer than normal time.
- % of time the page # is found in the associative registers is called hit ratio.

Page #	Frame #

Paging Hardware With TLB



Effective Access Time

- Associative Lookup = ε time unit, Assume memory cycle time is 1 microsecond
- **Hit ratio** – percentage of times that a page number is found in the associative registers; ratio related to number of associative registers
- Hit ratio = α

Effective access time=hit ratio*Associate memory access time +miss ratio* memory access time.

Example: If it takes 20 nsec to search the associative registers and **100 nsec to access** memory and hit ratio is 80 %, then, it takes 20 nanosecond to search the TLB. memory access takes 120nanosecond .-> 120+100 =220(total memory access time)

- **Effective access time=hit ratio*Associate memory access time +miss ratio* memory access time.**
- 0.80 * 120+0.20*220=140 nsec.
  - **40 % slowdown.**

- For 98-percent hit ratio, we have
  - Effective access time= 0.98*120+0.02*220
    = 122 nanoseconds
    = **22 % slowdown.**

Memory Protection

- Memory protection implemented by associating protection bit with each frame
- **Valid-invalid** bit attached to each entry in the page table:
- **"valid" indicates** that the associated page is in the process' logical address space, and is thus a legal page
- **"invalid" indicates** that the page is not in the process' logical address space

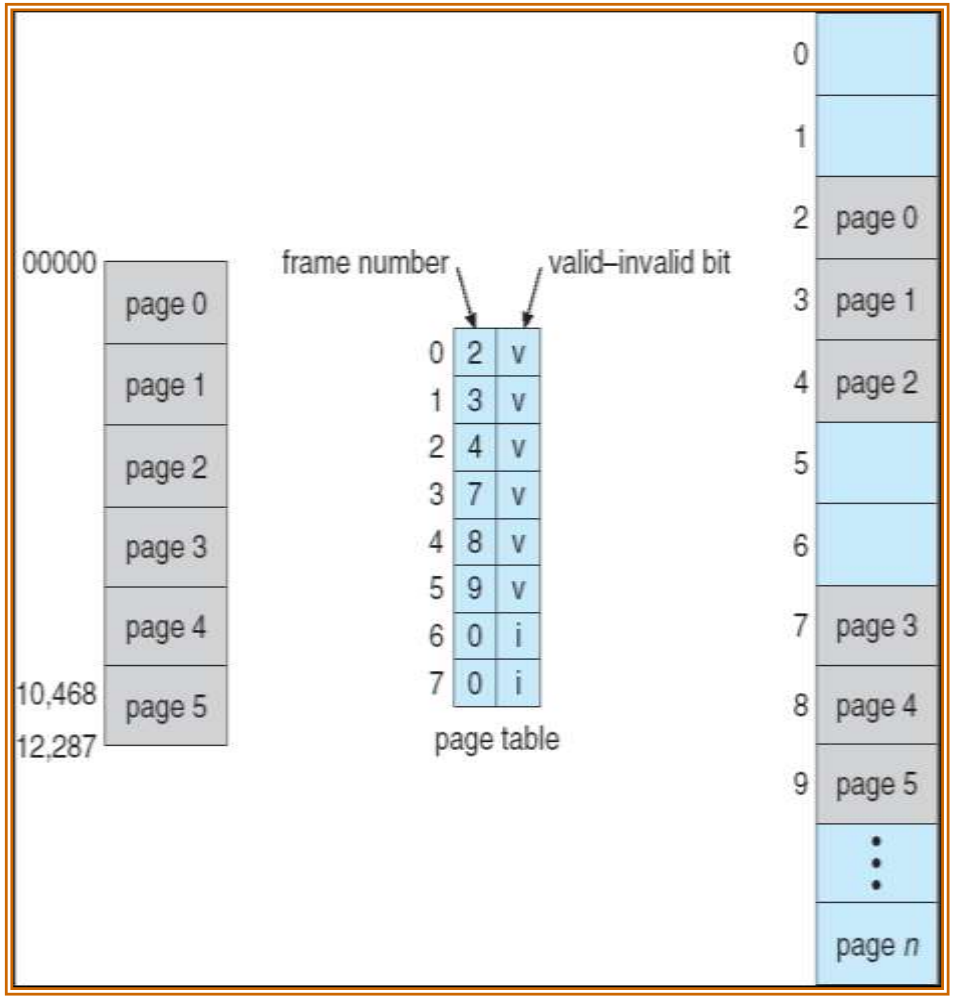
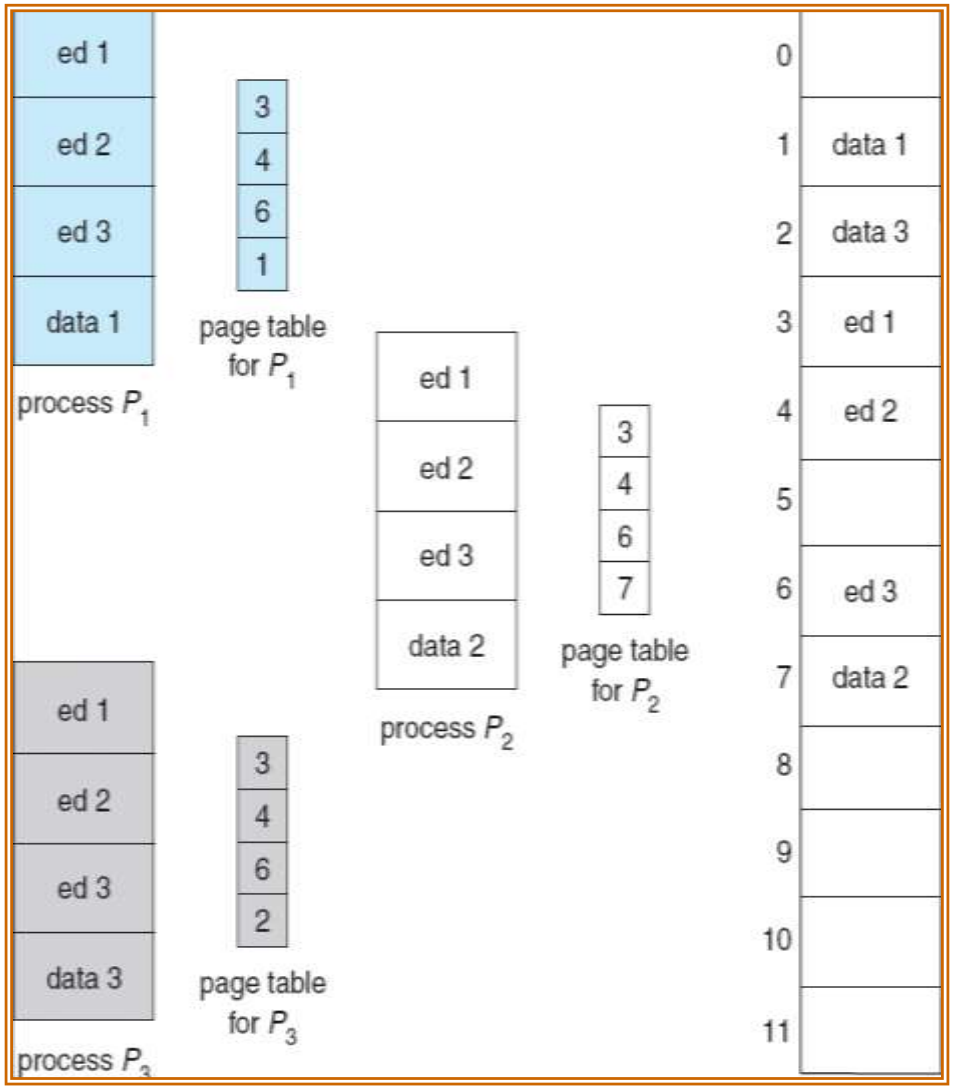


Figure:-Shared Pages Example

Shared Pages

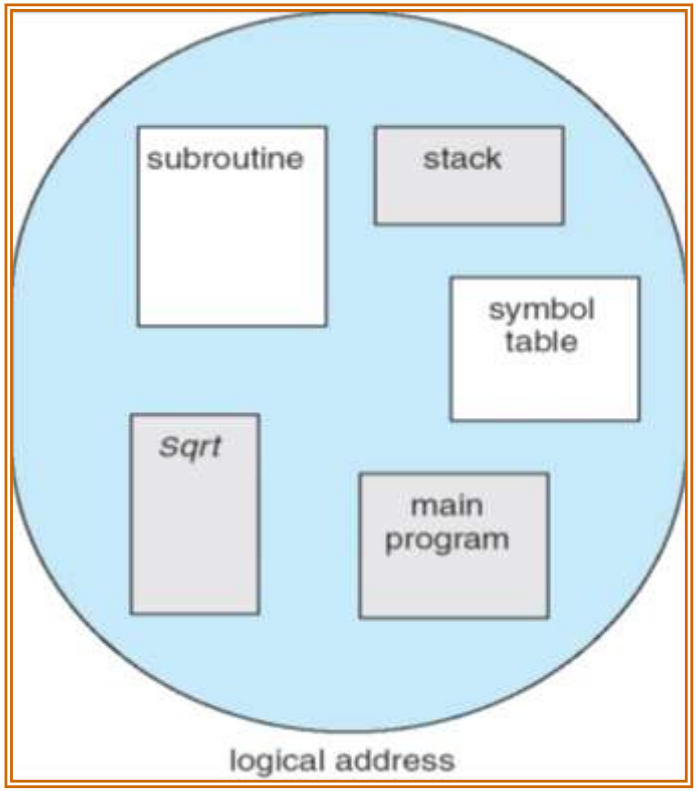
- **Shared code**
  - **One copy of read-only code shared among processes (i.e., text editors, compilers, window systems).**
  - Shared code **must appear in same location in the logical address space of all processes**
- **Private code and data**
  - Each process **keeps a separate copy of the code and data**
  - The pages for the private code and data can appear anywhere in the logical address space



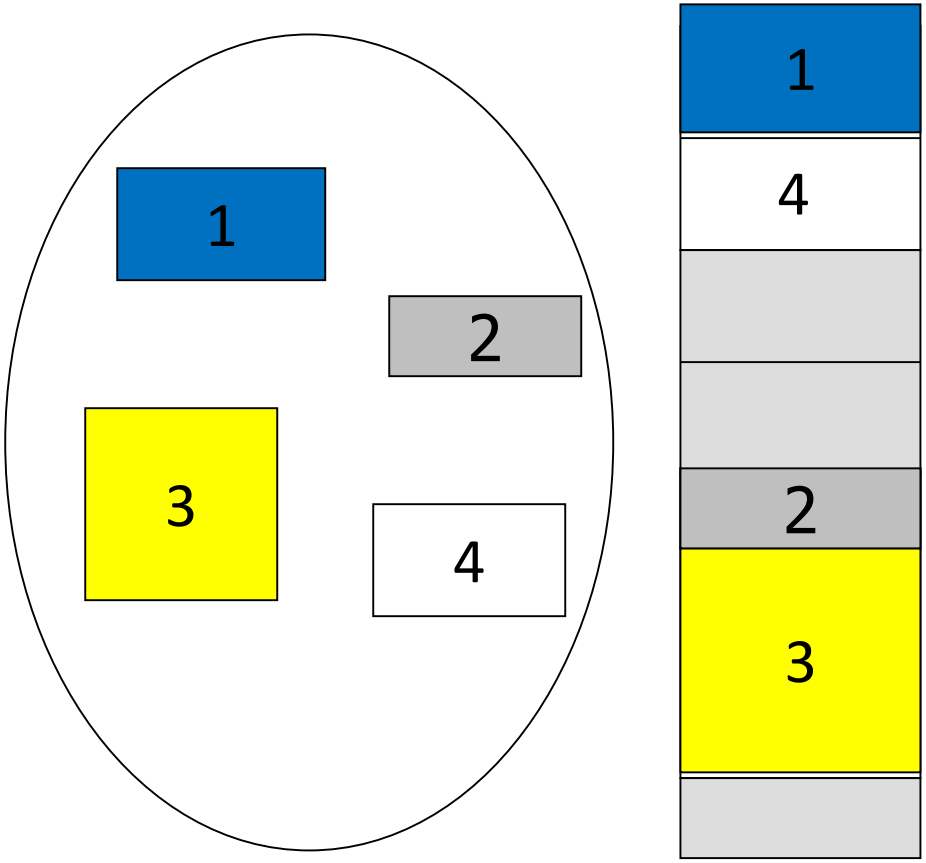
3. Segmentation

- Memory Management Scheme that supports user view of memory.
- A program is a collection of segments.
- A segment is a logical unit such as
    - main program, procedure, function
    - local variables, global variables, common block
    - stack, symbol table, arrays and so on.
  - Protect each entity independently
  - Allow each segment to grow independently
  - Share each segment independently

User's View of a Program



Logical View of Segmentation



user space

physical memory space

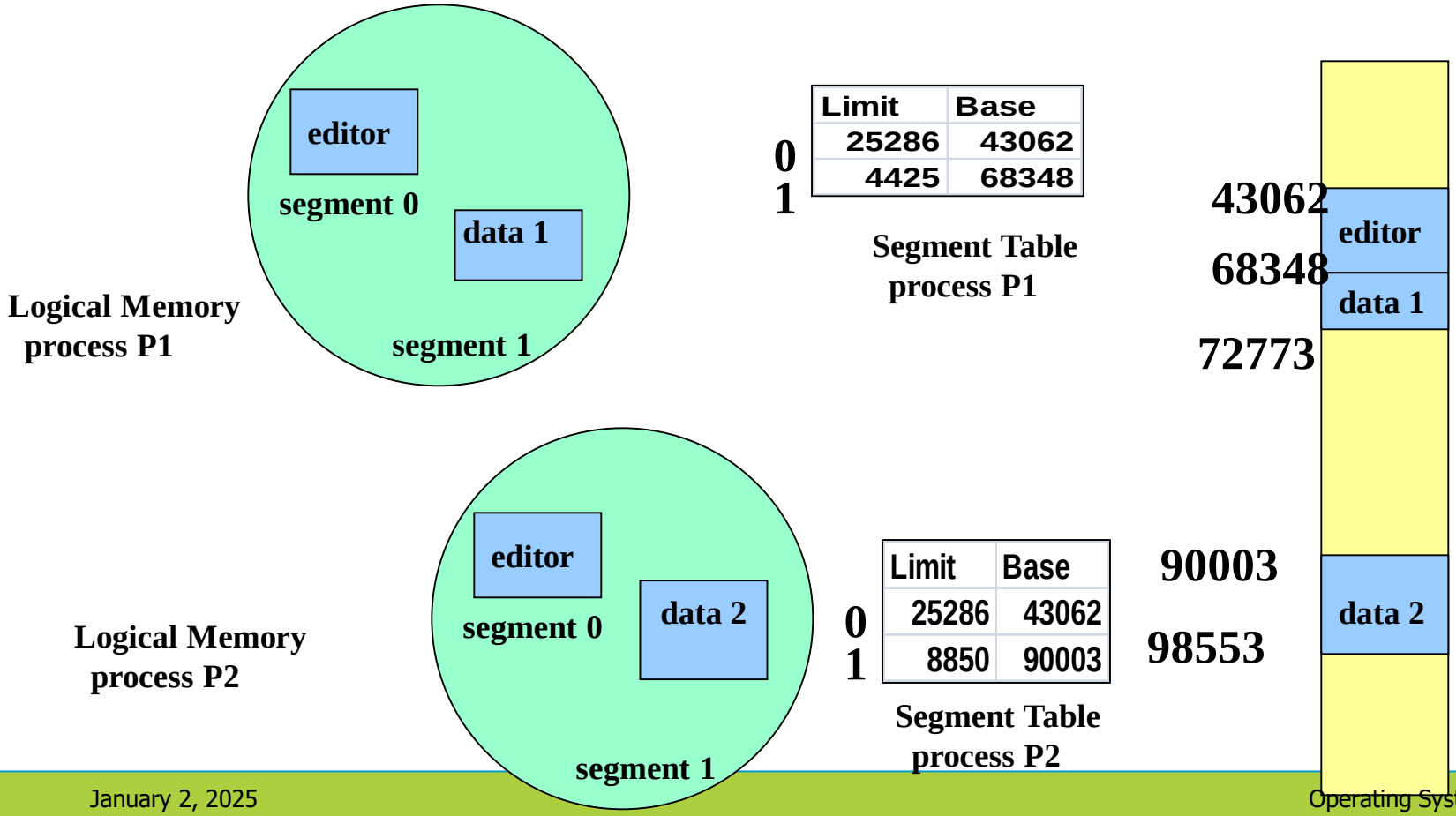
Segmentation Architecture

- **Logical address consists of a two tuple**
  - ✓ **<segment-number, offset>**
- Segment Table
  - Maps two-dimensional user-defined addresses into one-dimensional physical addresses. Each table entry has
    - **Base** - contains the starting physical address where the segments reside in memory.
    - **Limit** - specifies the length of the segment.
  - ***Segment-table base register*** (STBR) points to the segment table's location in memory.
  - ***Segment-table length register*** (STLR) indicates the number of segments used by a program;.
    - ❖ Note: ***segment number s is legal if s < STLR***.

Segmentation Architecture (cont.)

- **Relocation is dynamic - by segment table**
- **Sharing**
  - —Code sharing occurs at the segment level.
  - —Shared segments must have same segment number.
- **Allocation** - dynamic storage allocation problem
  - —use best fit/first fit, may cause external fragmentation.
- **Protection**
  - protection bits associated with segments
    - read/write/execute privileges
    - array in a separate segment - hardware can check for illegal array indexes.

Shared segments



Exercises

1) Consider a logical address space of 64 pages of 1,024 words each, mapped onto a physical memory of 32 frames.
   a. How many bits are there in the logical address?
   b. How many bits are there in the physical address?
2) Why are page sizes always powers of 2?
3) Describe a mechanism by which one segment could belong to the address space of two different processes.
4) Sharing segments among processes without requiring that they have the same segment number is possible in a dynamically linked segmentation system.
   a. Define a system that allows static linking and sharing of segments without requiring that the segment numbers be the same.
   b. Describe a paging scheme that allows pages to be shared without requiring that the page numbers be the same.

Exercises (…)

5) Given six memory partitions of 300 KB, 600 KB, 350 KB, 200 KB, 750 KB, and 125 KB (in order), how would the first-fit, best-fit, and worst-fit algorithms place processes of size 115 KB, 500 KB, 358 KB, 200 KB, and 375 KB (in order)? Rank the algorithms in terms of how efficiently they use memory.

6) Most systems allow a program to allocate more memory to its address space during execution. Allocation of data in the heap segments of programs is an example of such allocated memory. What is required to support dynamic memory allocation in the following schemes?
   a. Contiguous memory allocation
   b. Pure segmentation
   c. Pure paging

7) Consider a logical address space of 256 pages with a 4-KB page size, mapped onto a physical memory of 64 frames.
   a. How many bits are required in the logical address?
   b. How many bits are required in the physical address?

Exercises (…)

8) Compare the memory organization schemes of contiguous memory allocation, pure segmentation, and pure paging with respect to the following issues:
   a. External fragmentation
   b. Internal fragmentation
   c. Ability to share code across processes

9) Program binaries in many systems are typically structured as follows. Code is stored starting with a small, fixed virtual address, such as 0. The code segment is followed by the data segment that is used for storing the program variables. When the program starts executing, the stack is allocated at the other end of the virtual address space and is allowed to grow toward lower virtual addresses. What is the significance of this structure for the following schemes?
   a. Contiguous memory allocation
   b. Pure segmentation
   c. Pure paging